

[ C++\_ QT Day2 AI 사용보고서 ]

이름: 김도훈  
학번: 2026406401

## <목차>

## 1. 과제 1

(1)-1 과제1 AI 사용 내용1

## 2 과제 2

(2)-1 과제2 AI 사용 내용1

(2)-2 과제2 AI 사용 내용2

### 3. 깨달은 점

사용 LLM은 클로드 소넷입니다.

## 1. 과제 1

|                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>슬라이더를 처음 사용해보는 사람에게 설명하는 것 처럼 해주세요</p>                                                                                                                                                                                                                                                                                            | <pre>cpp connect(ui-&gt;slider, &amp;QSlider::valueChanged, this, &amp;MainWindow::onslideValueChanged);</pre>                                                                                                                                                                                                                                                                                                            |
| <p>2조 동안 생각할 수 있습니다</p> <p>슬라이더가 뭘지부터 아주 쉽게 설명드릴게요.</p> <p>슬라이더가 뭘까요?</p> <p>음악 플레이어에서 볼륨 조절할 때 옆으로 드래그하는 막대 본 적 있으시죠? 그게 슬라이더입니다. 손잡이를 좌우(또는 상하)로 움직이면 그 위치에 따라 숫자 값이 정해지는 위젯이에요. Qt에서는 이를 <b>QSlider</b> 라는 이름으로 제공합니다.</p> <p>왜 쓰나요?</p> <p>숫자를 직접 타이핑하지 않고도 마우스로 드래그만 해서 값은 정말 수 있게 해주는 게 목적입니다. 볼륨 밝기 속도 같은 걸 조절할 때 많이 씁니다.</p> | <p>이 한 줄이 하는 말을 풀어쓰면 이런겁니다. "slider 값이 바뀔 때마다 onslideValueChanged라는 함수를 실행해줘." 이 코드는 보통 MainWindow 생성자 안에 한 번만 써두면 됩니다.</p> <p><b>5단계: 실제로 무슨 일을 할지 정하기</b></p>                                                                                                                                                                                                                                                           |
| <p><b>1단계: 화면에 슬라이더 추가</b></p> <p>Qt Designer 화면 왼쪽에 위젯 목록이 쭉 나와있는데 그중에 Slider(가로 막대 모양 아예큰)를 찾아서 마우스로 끌어다가 창 안에 놓으면 끝입니다. 코드 한 줄도 안 쓰고 화면에 슬라이더가 생깁니다.</p>                                                                                                                                                                          | <pre>cpp void MainWindow::onslideValueChanged(int value) {     ui-&gt;valueLabel-&gt;setText(QString::number(value)); }</pre> <p>이 함수가 실제로 실행되는 부분입니다. 슬라이더가 바뀔 값이 <b>value</b> 라는 이름으로 자동으로 넘어옵니다. 그 값을 숫자에서 글자로 바꿔서(<b>QString::number</b>) 라벨에 표시해주는 거예요.</p>                                                                                                                                                          |
| <p><b>2단계: 이름 지어주기</b></p> <p>방금 놓은 슬라이더를 클릭하면 오른쪽에 속성 창이 뜹니다. 거기서 <b>objectName</b>이라는 항목에 원하는 이름을 적어주세요. 예를 들어 <b>slider</b> 라고 적으면 나중에 코드에서 <b>ui-&gt;slider</b> 라고 불러줄 수 있습니다.</p>                                                                                                                                               | <p><b>전체 흐름을 그림처럼 정리하면</b></p>                                                                                                                                                                                                                                                                                                                                                                                            |
| <p><b>3단계: 최소값과 최대값 정하기</b></p> <p>슬라이더는 기본적으로 0부터 99까지 움직입니다. 만약 0부터 100까지로 바꾸고 싶다면 속성 창에서 minimum을 0 maximum을 100으로 바꿔주면 됩니다. 이런 코드로 할 수 있어요.</p>                                                                                                                                                                                  | <pre>사용자가 슬라이더를 드래그함 ↓ 슬라이더가 "값 바뀌었어요!" 라고 신호를 보냄 (valueChanged) ↓ connect로 연결하면 함수가 자동으로 실행됨 (onslideValueChanged) ↓ 그 함수 안에서 라벨 텍스트를 새 값으로 바꿔줌</pre>                                                                                                                                                                                                                                                                    |
| <pre>cpp ui-&gt;slider-&gt;setRange(0, 100);</pre>                                                                                                                                                                                                                                                                                   | <p><b>꼭 기억될 새 가지</b></p>                                                                                                                                                                                                                                                                                                                                                                                                  |
| <p><b>4단계: 슬라이더를 움직이면 뭘가 반응하게 만들기</b></p> <p>여기가 제일 중요한 부분입니다. 슬라이더는 그 자체로는 그냥 숫자를 정하는 손잡이일 뿐이에요. 그 값이 바뀔 때 "뭘 할지"는 우리가 직접 정해줘야 합니다.</p> <p>Qt는 슬라이더 값이 바뀔 때마다 <b>valueChanged</b> 라는 신호(signal)를 자동으로 보냅니다. 우리는 그 신호를 받아서 실행할 함수(slot)를 하나 만들고 둘을 연결해주면 됩니다.</p>                                                                    | <ol style="list-style-type: none"> <li>슬라이더 자체는 값만 관리할 뿐 화면에 표시 하려면 라벨 같은 다른 위젯이 따로 필요함니다.</li> <li><b>connect()</b> 로 슬라이더와 함수를 미리 연결해줘야 값이 바뀔 때 뭘가 반응합니다. 이 연결을 안 하면 슬라이더는 움직이거만 하고 아무 일도 안 일어납니다.</li> <li>연결한 함수(<b>onslideValueChanged</b>)는 반드시 헤더 파일(mainwindow.h)의 <b>private slots:</b> 아래에 미리 선언되어 있어야 합니다.</li> </ol> <p>이 다섯 단계만 따라 하면 슬라이더 값이 라벨에 실시간으로 뜨는 걸 확인할 수 있어요. 직접 만들어 보실다가 막히는 부분 있으면 바로 물어보세요.</p> |
| <pre>cpp connect(ui-&gt;slider, &amp;QSlider::valueChanged, this, &amp;MainWindow::onslideValueChanged);</pre>                                                                                                                                                                                                                       | <p>📄 🗑️ 🔄 📄 📄 📄 📄</p>                                                                                                                                                                                                                                                                                                                                                                                                     |

1-(1) AI 사용 내용

LLM에게 정보를 제공받기전 구글링과 유튜브를 통해 정보를 얻기 위해 노력했다. 하지만 기초적인 사용법보다는 개념, 디자인과 같은 정보들이 많았다. 그래서 LLM에게 도움을 받았고 프롬프트를 입력할 때도 처음 사용하는 사람

에게 설명하는 것처럼 해달라고 입력했다. 그러니 기본적인 슬라이더 사용법, 슬라이더를 사용할 때 connect()를 어떻게 활용하는, signal과 slot이 어떻게 사용되는지 알려주었다. 이것을 통해 슬라이더 사용법도 알 수 있었지만 connect 함수 사용법 signal과 slot이 어떻게 사용되는지도 이해할 수 있었다. 그리고 이해한 정보를 바탕으로 내 코드에 슬라이더를 성공적으로 구현할 수 있었다.

## 2. 과제2

2주 동안 성취할 >

스택 구현에 쓸 수 있는 함수들을 세 가지 방식으로 나눠서 정리해드릴게요. 지금까지 쓰신 `std::vector` 방식과 표준 `std::stack` 그리고 Qt 자체 제공 `QStack` 까지 비교해서 보면 이해가 쉽습니다.

1. `std::vector`로 스택 구현 (지금까지는 방식)

| 함수                       | 역할                                |
|--------------------------|-----------------------------------|
| <code>push_back()</code> | 벡터 맨 뒤에 값 추가 (스택의 push)           |
| <code>pop_back()</code>  | 벡터 맨 뒤 값 제거 (스택의 pop, 반환값 없음)     |
| <code>back()</code>      | 벡터 맨 뒤 값을 읽기만 함, 제거 안 함 (스택의 top) |
| <code>empty()</code>     | 벡터가 비어있는지 확인                      |
| <code>size()</code>      | 현재 저장된 개수 반환                      |
| <code>clear()</code>     | 전부 비우기                            |

```

cpp
std::vector<int> history;
history.push_back(10); // push
int top = history.back(); // top 확인
history.pop_back(); // pop
        
```

2. `std::stack` (C++ 표준 전용 스택 컨테이너)

<stack> 헤더가 필요하며 내부적으로는 vector나 deque를 감싸고 있지만 push/pop 전용 인터페이스에만 노출됩니다.

| 함수                   | 역할                                  |
|----------------------|-------------------------------------|
| <code>push()</code>  | 맨 위에 값 추가                           |
| <code>pop()</code>   | 맨 위 값 제거 (반환값 없음, top()으로 먼저 읽어야 함) |
| <code>top()</code>   | 맨 위 값 읽기 (제거 안 함)                   |
| <code>empty()</code> | 비어있는지 확인                            |
| <code>size()</code>  | 개수 확인                               |

```

cpp
#include <stack>
std::stack<int> s;
s.push(10);
int top = s.top();
s.pop();
        
```

vector 버전과 함수 이름만 다를 뿐 동작은 완전히 동일합니다. 다만 `std::stack`은 연덱스도 중간 값에 접근하는 게 아예 불가능하도록 막혀 있어서 "이건 스택으로만 쓰겠다"는 의도가 코드에 더 명확하게 드러납니다. vector는 [1]로 아무 위치나 접근 가능하다는 차이가 있습니다.

3. `QStack` (Qt 자체 제공, 잘 안 쓰이지만 참고용)

<QStack> 헤더가 필요하고 QVector를 상속받아 만든 Qt 전용 스택 클래스입니다.

| 함수                     | 역할                                            |
|------------------------|-----------------------------------------------|
| <code>push()</code>    | 맨 위에 값 추가                                     |
| <code>pop()</code>     | 맨 위 값을 제거하면서 그 값을 반환 (std::stack과 다르게 반환값 있음) |
| <code>top()</code>     | 맨 위 값 읽기                                      |
| <code>isEmpty()</code> | 비어있는지 확인 (Qt 스타일이러 empty()가 아니라 isEmpty())    |
| <code>size()</code>    | 개수 확인                                         |

```

cpp
#include <QStack>
QStack<int> s;
s.push(10);
int value = s.pop(); // 제거하면서 값도 바로 반환
        
```

정리 비교표

| 동작     | vector                   | std::stack                  | QStack                      |
|--------|--------------------------|-----------------------------|-----------------------------|
| 추가     | <code>push_back()</code> | <code>push()</code>         | <code>push()</code>         |
| 제거     | <code>pop_back()</code>  | <code>pop()</code> (반환값 없음) | <code>pop()</code> (반환값 있음) |
| 맨 위 확인 | <code>back()</code>      | <code>top()</code>          | <code>top()</code>          |
| 비어있는지  | <code>empty()</code>     | <code>empty()</code>        | <code>isEmpty()</code>      |

### (2)-1 AI 사용1

라벨에 텍스트를 출력하는 것(과제1에서 구글링 통해 정보를 얻음)은 과제1의 방식과 똑같이 구현하여 문제가 없었지만 라벨에 출력된 텍스트를 읽어오는 기능을 구현하지 못하는 문제가 발생했다. 이 문제를 해결하기 위해 LLM을 사용했다. LLM은 label에 출력된 텍스트를 읽어오는 것은 text() 함수로 읽어올 수 있다는 정보를 제공해 주었고 바로 코드로 구현했고 버튼으로 라벨에 출력한 텍스트를 이 함수를 사용하여 읽어올 수 있었다.

## (2)-2 AI 사용 내용2

C++에는 스택과 관련된 함수들이 만들어져 있다는 것을 알고 있었다. C++ 프로그래밍을 할 때는 구글링을 통해서도 관련 함수들과 예제들을 많이 참고할 수 있었지만 QT 스택 관련 함수들은 자료가 많이 부족했고 자료가 있어도 나의 코드의 의도와는 맞지 않았다. 그래서 LLM을 활용했다. LLM에게 이전에 내가 프로그래밍 하던 코드를 입력해주었고 스택 관련 함수들에 대해 물어보니 내 코드에서 활용할 수 있고 내가 이해하기 쉬운 예제들을 제공해주었다. 특히 `push()`, `push_back()` 등과 같은 정보들을 활용해 스택의 후입 선출 기능을 구현할 수 있었다.

## 3 깨달은 점

단순한 C프로그래밍, C++ 프로그래밍에서 문제가 생기거나 궁금점이 생겼을 때 보다 QT 같은 기능이 많은 UI제작 프레임워크를 사용할 때 LLM을 사용하는 것이 더 효과적이고 더 많은 정보를 얻을 수 있는 것 같다. 이렇게 생각한 이유는 단순한 프로그래밍은 하나의 기능을 알고 있으면 그것을 응용해서 다른 것들을 구현할 수 있지만 UI는 단독적인 기능들이 많고 이것을 또 융합해서 사용하거나 응용하는 것이 매우 많은 것 같다. 이번에 QT를 처음 배워 구글링을 통해 얻은 자료들을 이해하기 힘들었다. 그래서 단순한 사용법 등과 같은 정보들을 얻는데 LLM의 도움을 많이 받았다. 이제 어느 정도 이해를 한 상태이기 때문에 QT통해 UI를 제작할 땐 구글링 자료들로 이해하여 사용할 수 있도록 노력할 것이다.